

Chapter 62: Database Internals

Personal Notes

Every previous system design chapter used databases as if they were magic boxes. "Store this in Postgres." "Use Cassandra for the write-heavy path." "MySQL for metadata, Redis for hot data." But WHY? Why is Postgres good at reads but not massive writes? Why does Cassandra scale writes so well but need compaction? Why do most databases lose data if crashed just wrong? Answering these questions requires cracking open the database and looking at the storage engine — the code that actually reads and writes bytes to disk.

This chapter is a change of pace after twelve sprawling system designs. Instead of "design X," we're going a level deeper. Understanding B-trees, LSM trees, WAL, MVCC, and compaction transforms your intuition about system design — you'll immediately grasp why each database was chosen in the previous chapters, and you'll know how to justify your DB pick in interviews with real technical arguments rather than "because it scales."

1. Why This Matters

Every system design interview eventually asks: "which database would you use, and why?" The answer separates memorized candidates from ones with real understanding. Interviewers expect you to reason about:

- Read-heavy vs write-heavy workload (drives B-tree vs LSM choice)
- How updates are handled (in-place vs append-only)
- How data survives crashes (WAL, checkpoints)
- How concurrent transactions don't corrupt each other (MVCC, locking)
- The trade-off between space + write amplification + read latency

Every database is ultimately a STORAGE ENGINE + query layer:

Query language (SQL, CQL, DSL)



Query planner + executor



STORAGE ENGINE ← This chapter is about THIS layer



Disk (SSD/HDD)

Same query layer can use different storage engines:

MySQL: InnoDB (B-tree) vs MyISAM (older)

MongoDB: WiredTiger (B-tree + LSM hybrid)
Postgres: default is B-tree
Cassandra: pure LSM
RocksDB: pure LSM (embedded)
DynamoDB: B-tree (SSD-optimized variant)

2. Storage Engines — The Layer We're Studying

A STORAGE ENGINE is the component that answers three questions:

1. Given a key, how do I read its value FROM DISK quickly?
2. Given a key + value, how do I WRITE it to disk durably?
3. If the machine crashes mid-write, how do I RECOVER without data corruption?

Two dominant designs answer these questions differently:

Aspect	B-Tree	LSM Tree
Update strategy	In-place mutation on disk pages	Append-only, write to log then reorganize later
Read latency	Excellent — few disk seeks per query	Good but can require checking multiple levels
Write latency	Moderate — must find + update page	Excellent — pure appends
Space amplification	1× (near ideal)	2-3× during compaction
Write amplification	1-3× (multi-page updates)	5-30× (rewrites during compaction)
Databases	MySQL InnoDB, Postgres, DynamoDB, MongoDB	Cassandra, RocksDB, LevelDB, HBase, ScyllaDB

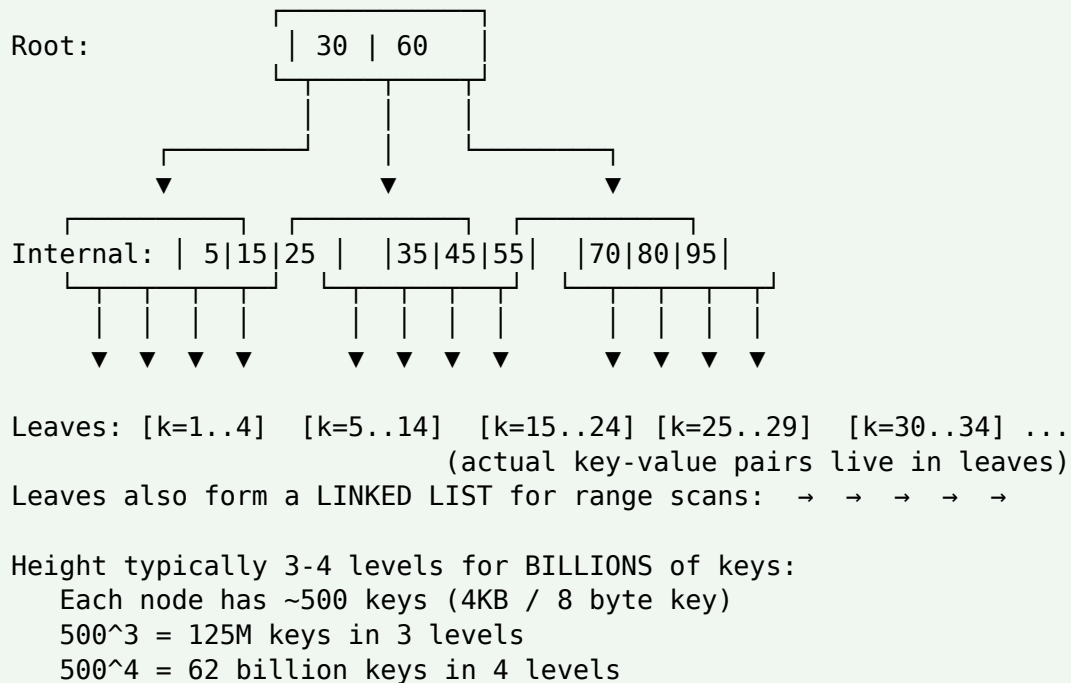
The pattern to internalize: B-tree databases are optimized for READS (in-place updates keep data organized). LSM tree databases are optimized for WRITES (append-only avoids expensive random writes). Everything else — space, latency, compaction — is a consequence of this fundamental split. When you know the workload, you can predict the storage engine.

3. B-Trees — The Workhorse

B-trees have been the dominant database index structure since the 1970s. Every major SQL database uses them. They generalize binary search trees to have MANY keys per node (typically hundreds) — matching the physical reality that disk reads happen in blocks of ~4-16 KB.

The Structure

A B+Tree with pages (blocks) instead of individual nodes:



Read Path — $O(\log_B N)$ Disk Reads

- Start at root (usually cached in RAM)
- Binary search within the root's ~500 keys → decide child
- Follow pointer, read the child page (~1 disk seek)
- Repeat until leaf reached
- For 1B keys: 3-4 disk seeks total → 10-30 ms on HDD, <1 ms on SSD

Write Path — In-Place Update

4. Find the leaf page where key belongs
5. Update the page in memory (buffer pool)
6. Write to WAL first (crash safety — see section 5)
7. Mark page as "dirty"; flush to disk later (checkpointing)

The Splitting Problem

Leaf page can hold 500 keys. What happens when it's full?

Before insert of key=42 (leaf full):

[30, 31, 32, ..., 45, 46, 47] ← 500 keys, sorted

Split into two pages:

```
Left: [30..37]      (250 keys)
Right: [38..47]     (250 keys)
Parent gets new pointer to right page.
```

If parent is full too? Recursively split parents up to root.
If root splits? Tree grows by one level (rare – happens as DB grows).

This is where B-tree write amplification comes from –
one logical write can rewrite multiple pages up the tree.

B-trees update disk pages IN PLACE: This is fast for read-heavy workloads (data stays organized) but painful for writes. Random writes on SSDs cause write amplification at the flash layer (rewriting entire flash blocks). This is why write-heavy workloads often pick LSM instead — sequential appends are much friendlier to storage hardware.

4. LSM Trees — The Write-Optimized Alternative

Log-Structured Merge trees flip the B-tree strategy: don't update pages in place. Instead, always APPEND. New writes go to an in-memory buffer that gets FLUSHED to disk as an immutable sorted file. Older files are periodically MERGED to keep the number of files manageable. Invented in 1996; became mainstream via Google's Bigtable and later Cassandra, RocksDB, LevelDB.

The Structure

Writes flow like this:

```
Client → WAL (durability) → MEMTABLE (in-memory sorted map)
                               |
                               | (when full, ~64 MB)
                               ▼
                        FLUSH to disk as SSTABLE
                        (Sorted String Table – immutable file)
```

Disk state – a stack of SSTables in levels:

Level 0 (fresh, unsorted between files):
SST_a SST_b SST_c (each ~64 MB, from memtable flushes)

Level 1 (compacted, sorted, non-overlapping):
SST_d[a-k] SST_e[l-z]

Level 2 (larger, non-overlapping):
SST_f[a-e] SST_g[f-m] SST_h[n-r] SST_i[s-z]

Level 3+ (each level ~10× larger than previous):
...

Each level: files are sorted; higher levels are larger.

Write Path — Extremely Fast

8. Write to WAL (durability, sequential append)
9. Insert into in-memory memtable (typically a skip-list or red-black tree)
10. Return success — DONE. Total time: microseconds.
11. Later, background thread flushes memtable to disk as immutable SSTable

Read Path — Can Get Complex

```
get(key):  
    # Check in reverse chronological order — newest wins  
    1. Look in memtable (fast, in RAM)           → found? return.  
    2. Look in Level 0 SSTables (may overlap)    → found? return.  
    3. Look in Level 1 SSTable containing key    → found? return.  
    4. Look in Level 2 SSTable containing key    → found? return.  
    ...  
    N. Look in deepest level                     → still not found? return  
    NULL.
```

Worst case: check ~10 SSTables (across ~7 levels) — 10 disk seeks.
That's SLOW compared to B-tree's 3-4 seeks.

Optimizations:

- Bloom filter per SSTable → skip files that definitely don't have the key
- Block cache → cache hot SSTable blocks in RAM
- Level-scoped indexing → jump directly to the right SSTable in a level

Deletion — Tombstones

You can't just remove data from an SSTable — the file is IMMUTABLE. Instead, write a TOMBSTONE ("key X is deleted at time T"). Reads that see a tombstone return "not found." Tombstones get physically removed only during compaction, when the tombstone plus all older values can safely be dropped.

5. Compaction — The LSM Tax

Without compaction, LSM trees would accumulate infinite SSTables — reads would slow to a crawl. COMPACTION is the background process that merges SSTables from a lower level into a higher level, removing duplicates + tombstones along the way. It's LSM's most important operational concept.

Two Strategies

Strategy	Approach	Trade-off
Size-tiered	Merge equally-sized SSTables into one bigger	Simple; higher write amplification

Strategy	Approach	Trade-off
Leveled	Levels grow by 10×; L1 files never overlap	Cassandra + RocksDB default

Cost of Compaction

- WRITE AMPLIFICATION — one logical write may be rewritten 10-30× as data migrates through levels
- BACKGROUND I/O LOAD — compaction competes with user reads/writes for disk bandwidth
- SPACE AMPLIFICATION — old + new SSTables coexist briefly during compaction

Compaction is why LSMs "feel slow" sometimes: A well-tuned LSM performs beautifully during steady state. But under sudden write bursts, compaction falls behind — read amplification grows (many SSTables to check), disk fills up, and eventually the system becomes unresponsive. Operating LSMs (Cassandra, RocksDB) is largely about MANAGING compaction: sufficient CPU, tuned rate limits, monitoring for backlog.

6. WAL — The Foundation of Durability

The Write-Ahead Log is the single most important database concept for correctness. Both B-trees and LSMs use it. The rule: BEFORE modifying any on-disk data structure, write the intended change to a sequential log. If the process crashes, recovery replays the log from the last checkpoint. Nothing is ever "lost in flight."

How It Works

Write flow with WAL:

Client: INSERT ... VALUES (...)

- ```

 |
 ▼
1. Append to WAL file (sequential write) ← FAST (sequential I/O)
 WAL entry: {"op":"insert", "table":"users", "row":{"...}, "lsn":42}
 |
 ▼
2. fsync() the WAL to actually persist to disk
 (Without fsync, entry sits in OS buffer – lost on crash!)
 |
 ▼
3. Return SUCCESS to client ← Client sees "committed"
 |
 ▼
4. LATER (background): apply the change to actual data pages
 Update B-tree pages, or add to memtable for LSM.

```

Recovery after crash:

- Find last checkpoint (say, LSN=40 was fully flushed)
- Replay WAL from LSN=41 onwards
- System is back where it was pre-crash.

## fsync — The Real Villain

The fsync() system call is what actually forces bytes to physical disk. WITHOUT it, writes sit in OS memory and are lost on power failure. WITH fsync on every write, throughput is limited to disk IOPS. Databases balance this with GROUP COMMIT: batch many transactions' WAL entries together, fsync once, ack them all. This is the #1 performance knob in traditional databases.

```
// Naive approach: fsync per commit
begin_txn(); ... ; fsync(wal); commit(); // 1 fsync per txn = ~200/sec

// Group commit: many txns share one fsync
1. Threads T1, T2, T3 each append to WAL
2. First thread (T1) triggers fsync
3. T2 and T3 wait for T1's fsync to complete
4. All three commit together
Result: ~10,000+ txns/sec with the same durability guarantee.
```

**This concept extends to distributed systems:** Raft's log (Ch 63 preview) is a distributed WAL. Kafka topics are essentially durable WAL streams. Amazon Aurora replaces the B-tree write path entirely with a WAL that ships to distributed storage. WAL is one of those primitives that pops up in every serious distributed data system.

## 7. MVCC — How Databases Handle Concurrent Transactions

You've got two users reading + writing the same row. How does the database prevent chaos? Old databases used LOCKS — readers block writers, writers block readers. Modern databases use MVCC (Multi-Version Concurrency Control): every row has multiple VERSIONS; readers see the version that was current when THEIR transaction started. No blocking.

### The Idea

Row: (user\_id=42, balance=100), created at TX\_ID=10

Transaction TX\_15 starts, reads: balance=100  
(sees version created at TX\_ID=10)

Meanwhile TX\_16 starts, updates balance to 200  
Row now has TWO VERSIONS in storage:  
Version 1: balance=100, valid TX\_ID [10, 16)  
Version 2: balance=200, valid TX\_ID [16, ∞)  
TX\_16 commits.

TX\_15 (still running) reads balance again:

Sees Version 1 (still 100) – CONSISTENT SNAPSHOT  
Not affected by TX\_16's write. No blocking.

After TX\_15 completes, Version 1 becomes garbage – VACUUM cleans up.

## Two MVCC Implementations

- Postgres — appends new tuple version, marks old as dead; VACUUM cleans dead tuples
- MySQL InnoDB — updates row in place, keeps old version in the UNDO LOG for readers

## Snapshot Isolation

The most common MVCC isolation level. Every transaction sees a SNAPSHOT of the database as of the start of the transaction. All the transaction's reads are consistent with each other — even if another transaction commits changes during that time. This is what "REPEATABLE READ" gives you in Postgres and MySQL.

**Snapshot isolation ≠ serializable:** Snapshot isolation is NOT the strongest isolation. It allows a subtle anomaly called "write skew" where two transactions each see valid state but commit contradictory writes. For example: two doctors each check "is another doctor on call?" (both see yes), each decides they can go off duty. Result: no doctors on call. Postgres offers SERIALIZABLE isolation on top of MVCC for this — with detection + retry.

# 8. Isolation Levels — What You Actually Get

SQL defines four isolation levels. Understanding them matters because interviewers ask, and picking wrong causes real bugs.

| Level            | What It Guarantees                                     | When To Use                                             |
|------------------|--------------------------------------------------------|---------------------------------------------------------|
| Read uncommitted | Reads may see uncommitted data (dirty reads)           | Rarely used — anomalies too severe                      |
| Read committed   | Only committed data visible; but rows change under you | Postgres default — 'good enough' for many apps          |
| Repeatable read  | Same row reads return same value throughout txn        | MySQL default; Postgres calls this 'snapshot isolation' |
| Serializable     | As if all txns ran one at a time in some order         | Strongest; slowest; detects + retries anomalies         |

**For payments (Ch 60) and trading (Ch 61):** Use SERIALIZABLE. Financial invariants (double-entry balance, position quantities) can't tolerate write skew anomalies. Yes, it's 20-30% slower — worth it for correctness. For social feeds, catalogs, and other non-critical data, Read Committed is fine.



## 9. Indexes — The Speed Layer

An INDEX is a separate data structure that speeds up lookups on non-primary keys. Every serious database supports secondary indexes — but their implementation varies dramatically between B-tree and LSM engines.

### Primary vs Secondary Indexes

- PRIMARY INDEX — the main storage of the table, keyed by primary key
- SECONDARY INDEX — a separate structure mapping (index\_key → primary\_key)

```
Table: users
 primary key: user_id
 columns: user_id, email, name, created_at

Primary index (B-tree/LSM keyed by user_id):
 user_id → {email, name, created_at, ...}

Secondary index on email (separate B-tree/LSM):
 email → user_id (points back to primary)

Query: SELECT * FROM users WHERE email = 'a@b.com'
 1. Look up email 'a@b.com' in secondary index → user_id=42
 2. Look up user_id=42 in primary index → full row
 Two lookups instead of a full table scan.
```

### Covering Indexes

A COVERING INDEX includes all columns the query needs, so no lookup back to the primary index is required. Trade-off: bigger index size, faster reads. Postgres's INCLUDE clause and MySQL's composite indexes support this.

### Indexes in LSM Systems

Secondary indexes are HARDER in LSM databases because updating an index atomically with the primary data is tricky (both live in immutable SSTables). Cassandra's local secondary indexes have well-known scale limits; that's why apps often maintain their own denormalized indexes via async pipelines instead. This is a subtle but important gotcha.

## 10. Bloom Filters — Making LSMs Fast

We covered Bloom filters in Ch 58 (crawler dedup). They're equally critical inside LSM databases. Every SSTable has a Bloom filter that answers "could this SSTable POSSIBLY contain this key?" instantly, without reading the file. False positives are rare (~1%). False negatives never happen. This is what makes LSM reads competitive with B-trees.

```
Read for key=42 in LSM database (7 levels):
```

Without Bloom filters:

Check every level's SSTable → ~10 disk seeks

With Bloom filters:

Level 0: 3 SSTables. Check each Bloom → 2 say "no", 1 says "maybe"  
→ Check that 1 → not found

Level 1: Bloom says "no" → skip

Level 2: Bloom says "no" → skip

Level 3: Bloom says "yes" → check → FOUND

Total: 2 real disk seeks (vs 10 without Bloom).

Bloom filters make LSM reads ~5× faster.

**This isn't optional in LSM:** Every serious LSM database — RocksDB, Cassandra, LevelDB, HBase — uses Bloom filters aggressively per SSTable. Sized to ~10 bits per key for 1% false positive rate. Without them, LSMs would be much slower on reads and B-trees would dominate. It's an integral part of the design, not a bolt-on optimization.

## 11. Real-World Databases Mapped to These Primitives

| Database     | Engine  | Concurrency            | Best For                         |
|--------------|---------|------------------------|----------------------------------|
| PostgreSQL   | B-tree  | MVCC + Snapshot        | OLTP, complex queries, JSON      |
| MySQL InnoDB | B-tree  | MVCC + undo log        | OLTP, e-commerce, most SaaS      |
| Cassandra    | LSM     | Row-level, no txns     | Write-heavy time-series, IoT     |
| RocksDB      | LSM     | Point (no cross-key)   | Embedded storage engine          |
| DynamoDB     | B-tree* | Item-level cond writes | AWS-scale KV, consistent hashing |
| MongoDB (WT) | B-tree  | Snapshot               | Document store, JSON-heavy       |
| HBase        | LSM     | Row-level              | Hadoop ecosystem, write-heavy    |
| ScyllaDB     | LSM     | Row-level, no txns     | Cassandra-compatible, C++ speed  |

## 12. Decision Framework — Which Database?

Now that you know the internals, DB selection becomes principled. Here's a decision framework:

### Ask These Questions

12. Read : Write ratio? Read-heavy (>90% reads) → B-tree. Write-heavy → LSM.
13. Do you need transactions? Financial → YES (Postgres, MySQL). Time-series → often no (Cassandra).
14. What's the working set size? RAM-fit → B-tree. Much bigger than RAM → LSM's write efficiency matters more.
15. Do you need secondary indexes? Yes + strong consistency → B-tree. LSM has caveats.
16. What's your access pattern? Point lookups + small ranges → both work. Big scans → LSM is often faster (sequential).

### Common Choices for the Previous Chapters

- Ch 51 (URL shortener): B-tree (Postgres/DynamoDB) — read-heavy, small working set
- Ch 52 (Feed): B-tree for metadata + Redis for hot cache — reads dominate
- Ch 53 (Chat): LSM (Cassandra) for message store — writes dominate
- Ch 55 (YouTube): B-tree for metadata (Postgres) + blob store for video
- Ch 57 (Distributed Cache): pure in-memory (Redis/Memcached), no persistent engine
- Ch 58 (Crawler): LSM (Cassandra) for URL metadata — write-heavy discovery
- Ch 60 (Payments): B-tree (Postgres) for the ledger — needs transactions
- Ch 61 (Trading): in-memory book + LSM (Cassandra) for trade history — hybrid

## 13. Common Misconceptions

### Misconception 1: NoSQL is always faster

NoSQL (LSM) is faster for WRITES. It's often SLOWER for reads. Choose based on workload, not hype. Postgres is remarkably fast — modern versions handle millions of transactions per second on beefy hardware.

### Misconception 2: LSM avoids random I/O

LSM WRITES are sequential. LSM READS may involve random I/O across multiple SSTables. That's what Bloom filters mitigate — but they don't eliminate it.

### Misconception 3: Bigger indexes are always faster

More indexes → more disk space + slower writes (every insert updates each index). Only index what you actually query on. Unused indexes are pure cost.

#### Misconception 4: fsync is always slow

Group commit batches many transactions into one fsync. Modern SSDs handle ~100K+ fsyncs/sec. What kills throughput is fsync-per-request without batching.

#### Misconception 5: MVCC eliminates locks entirely

Even in MVCC databases, WRITERS still need to serialize their writes. Two updates to the SAME row must be ordered. Locks still exist under the hood — just usually short-lived.

#### Misconception 6: The DB is a black box

Every DB tuning knob (buffer pool size, WAL settings, compaction rate, autovacuum) makes sense once you understand the internals. Treating the DB as a black box guarantees you'll misuse it. Read the manual.

## 14. Best Practices

- PICK THE ENGINE BY WORKLOAD — B-tree for reads/transactions, LSM for writes/time-series
- USE SERIALIZABLE isolation for money; Read Committed for everything else
- MONITOR write amplification — a well-tuned system has WA of 5-10× (LSM) or 2-3× (B-tree)
- SIZE YOUR BUFFER POOL — enough RAM to hold hot pages; typically 25-50% of DB size ideal
- ADD BLOOM FILTERS to LSM tables — non-optional for read performance
- USE GROUP COMMIT for high-throughput workloads — massively reduces fsync overhead
- VACUUM/COMPACT regularly — don't let dead tuples or L0 SSTables accumulate
- MEASURE, don't assume — every workload is different; benchmark on production-like data
- KNOW YOUR ISOLATION LEVEL — most bugs come from misunderstanding what your DB guarantees
- LOG QUERIES that use full table scans — an unindexed hot query kills performance

## 15. Quick Reference

| Concept        | Meaning                                                             |
|----------------|---------------------------------------------------------------------|
| Storage engine | Component that reads/writes bytes on disk (below query layer)       |
| B-tree         | Sorted tree of pages, updates in place — read-optimized             |
| LSM tree       | Append-only log of SSTables, merged in background — write-optimized |
| SSTable        | Sorted String Table — immutable disk file in LSM                    |
| Memtable       | In-memory buffer holding recent writes before flush                 |
| WAL            | Write-Ahead Log — sequential durability record                      |

| Concept                       | Meaning                                                         |
|-------------------------------|-----------------------------------------------------------------|
| <b>fsync</b>                  | Force OS buffer contents to physical disk                       |
| <b>Group commit</b>           | Batch many txns into one fsync — huge throughput win            |
| <b>Compaction</b>             | LSM's merging of SSTables to reclaim space + reduce read cost   |
| <b>Write amplification</b>    | Physical writes ÷ logical writes — 1× ideal, 10-30× LSM typical |
| <b>MVCC</b>                   | Multi-Version Concurrency Control — readers don't block writers |
| <b>Snapshot isolation</b>     | Each txn sees a consistent snapshot from its start time         |
| <b>Serializable isolation</b> | Strongest — as if txns ran one at a time                        |
| <b>Write skew</b>             | MVCC anomaly not prevented by snapshot isolation                |
| <b>Tombstone</b>              | Deletion marker in an immutable file (LSM)                      |
| <b>Bloom filter</b>           | Probabilistic set for 'is X in this SSTable?' — vital in LSM    |
| <b>Covering index</b>         | Index with all query columns — no lookup back to primary        |
| <b>Vacuum (Postgres)</b>      | Removes dead tuples from MVCC storage                           |

## 16. What Interview Questions This Unlocks

With these primitives, you can answer any DB-selection or DB-behavior question:

17. "Why did you pick Cassandra over Postgres for this?" → write-heavy workload, LSM's sequential-append advantage
18. "What happens if the database crashes mid-transaction?" → WAL replay from last checkpoint
19. "Why is the DB slow after a burst of writes?" → compaction backlog (LSM) or dirty page flushes (B-tree)
20. "How does Postgres handle 1000 concurrent readers?" → MVCC — no locking on reads
21. "Why does your write throughput plateau?" → fsync bottleneck; look at group commit
22. "Why is Cassandra fast for reads sometimes and slow other times?" → SSTable count varies; compaction state matters
23. "How would you design a KV store from scratch?" → LSM: memtable + WAL + SSTables + compaction + Bloom filters
24. "What isolation level would you use for this financial app?" → Serializable; explain write skew

## 17. Final Takeaway

Understanding B-trees and LSM trees changes how you see every system design. Every database is one of these two engines wrapped in a query language + distribution layer. When someone says "we use Cassandra," you now know that means: LSM tree, memtable + WAL + SSTables, tunable per-row consistency, no cross-row transactions, secondary index caveats, needs compaction tuning, benefits from Bloom filters. That's a much richer mental model than "NoSQL that scales."

For interviews, the FIVE concepts to internalize are: (1) B-tree = in-place, read-optimized; (2) LSM = append-only, write-optimized; (3) WAL = the foundation of durability; (4) MVCC = how multiple readers/writers coexist without blocking; (5) compaction = the LSM tax that must be tuned. With these five, you can reason about ANY database choice, defend ANY design decision, and diagnose ANY performance issue in interviews or production.

**Key Takeaway:** Databases = STORAGE ENGINE + query layer. Two dominant engines: B-TREE (in-place updates, read-optimized — Postgres, MySQL, DynamoDB) vs LSM TREE (append-only, write-optimized — Cassandra, RocksDB, HBase). Both use WAL for durability. LSM adds MEMTABLE + SSTables + COMPACTION + BLOOM FILTERS. Concurrency handled via MVCC (readers don't block writers) with snapshot or serializable isolation. Choose engine by workload: read-heavy → B-tree, write-heavy → LSM. Choose isolation by criticality: money → serializable, everything else → read committed. Monitor write amplification + compaction backlog + buffer pool hit rate.

## 18. What's Next?

Databases internals done — the foundation is set. Next deep dives + designs:

- Deep Dive: Kafka Architecture — partitions, consumer groups, exactly-once semantics
- Deep Dive: Consensus Algorithms — Raft (worked example), Paxos, leader election
- Deep Dive: Distributed Transactions — 2PC, sagas, event sourcing (extend Ch 60)
- Design a Notification System — cross-channel delivery, priorities, batching
- Design a Metrics/Monitoring System (Datadog-style) — time-series data
- Design a Distributed Job Scheduler — Cron at scale
- Design a Feature Flag / A/B Testing System
- Design a Log Aggregation Service (Splunk-style)